Project overview

Stock Option Analyzer

AdvComPro-Mini-Project-2026

Project	Submission details
Group	Marlon and Friends
Document	System design and implementation specification
Prepared	24 October 2026
Submission deadline	31 October 2026
Build scope	Python/FastAPI + Next.js Docker application in the current project folder

Student ID	Team member
68011436	Panupon Manowattana
68011286	Arood Parntharm
68011446	Pathompong Mala
68011282	Apiwit Seephoto

Executive summary

Stock Option Analyzer is an educational web application that combines stock-market context, AI-generated explanations and transparent probability calculations. A user selects a ticker and analysis horizon, reviews bullish/bearish reasoning and CALL/PUT explanations, and can inspect saved results, scan predefined watchlists and consult an investment glossary.

The system separates the browser interface, application API and persistent database. Finnhub supplies market context; the backend sends selected context to Anthropic and enriches the returned JSON with local option-level calculations. Free and Pro accounts have different usage and feature access. Membership requests are reviewed manually by an administrator.

Document boundary

This document describes the code inspected for this submission. It distinguishes implemented behavior from acceptance tests and future work. It does not claim predictive accuracy, automatic payment processing, live option-chain pricing or a verified university-server deployment.

DESIGN DOCUMENT / 02

Problem, users and requirements

Problem and objectives

Stock news, price data, terminology and option calculations are usually spread across separate tools. This project brings them into one learning workflow, preserves analysis history and makes the assumptions behind probability estimates visible. Success means that a reviewer can reproduce the application, understand its data flow and distinguish model output from market facts.

Actor	Main use cases
Visitor	Read the interface and glossary; register or sign in.
Free member	Analyze up to three successful requests per UTC day, review own history and use Free watchlists.
Pro member	Analyze without the Free daily quota and unlock Pro watchlists while membership remains valid.
Administrator	Review membership requests, approve 30-day Pro access, and manage flash-sale records using the configured admin credential.

ID	Functional requirement	Implementation
FR-01	Register, sign in, identify current user	auth.py; main.py auth routes
FR-02	Analyze ticker and selected horizon	POST /api/analyze; AnalysisDetails
FR-03	Explain direction, factors, news and option strategy	AI JSON + result panels
FR-04	Calculate option levels and strike probabilities	probability.py + browser calculator
FR-05	Save and retrieve user-specific history	analysis_history; latest 20 rows
FR-06	Scan predefined Free/Pro watchlists	scanner.py; scanner page
FR-07	Look up investment terms in English	glossary.py; glossary page
FR-08	Request and administer Pro access / sales	subscriptions; flash_sales; admin page

Quality requirements and scope

Maintain clear module boundaries, persistent PostgreSQL data, explicit loading/error states and configuration outside source code. The intended reviewer workflow runs with Docker Desktop. External-provider availability and latency remain dependencies. Broker execution, automated settlement, backtesting and guaranteed trading outcomes are outside the implemented scope.

DESIGN DOCUMENT / 03

Architecture and module design

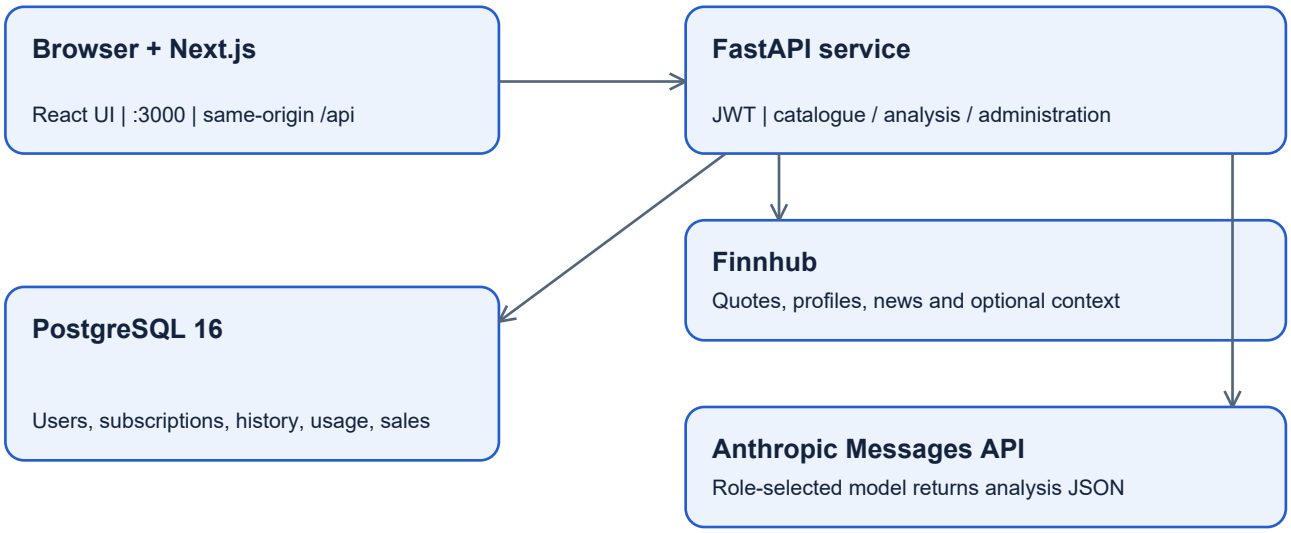


Figure 1. Logical runtime architecture. Arrows denote requests or data access; they are not deployment-verification claims.

Layer	Responsibilities and boundaries
Browser / Next.js	app/ routes render the six feature pages. lib/api.ts adds the stored bearer token to requests. /api rewrites forward through the frontend service to FastAPI.
FastAPI / application	main.py defines routes and orchestrates analysis and persistence. auth.py handles hashing, token validation and membership expiry. schemas.py defines input and selected output models.
Domain modules	finnhub_client.py normalizes market context; probability.py implements numeric estimates; scanner.py defines watchlists and heuristics; glossary.py provides learning terms.
Persistence	database.py defines SQLAlchemy models and scoped sessions. PostgreSQL stores business records; a named Docker volume retains database files across container restarts.
External systems	Only the backend needs Finnhub and Anthropic API keys. The browser receives results, not configured provider credentials.

Deployment topology

docker-compose.yml runs db, backend and frontend services. Database and backend host ports bind to 127.0.0.1; the local frontend also binds to loopback. Service-to-service database access uses db:5432, and the frontend proxy uses BACKEND_URL=http://backend:8000 rather than container-local localhost.

docker-compose.server.yml exposes frontend port 3000 on the host while retaining loopback bindings for PostgreSQL and FastAPI. The configured university machine is a possible target, not proof of an active deployment. A reverse proxy, HTTPS and a restricted network policy are production follow-up work.

DESIGN DOCUMENT / 04

Analysis workflow and data contract

Step	Interaction / processing
1. Request	Browser submits {ticker, market, horizon} to POST /api/analyze with Authorization: Bearer token.
2. Access check	Resolve the user; downgrade expired Pro membership when resolving authenticated requests. Free accounts are blocked at three completed analyses per UTC day.
3. Market context	Fetch Finnhub quote, profile, news and available recommendations, sentiment and upcoming earnings. Optional fields can be absent.
4. AI request	Build a prompt using selected context and horizon. Pro selects the configured Opus model identifier; Free selects Haiku. A single Messages API call requests JSON.
5. Enrichment	Parse the response JSON, obtain estimated IV, calculate option_levels and attach days_left, normalized stock_data and model_used.
6. Transaction	Increment daily_usage and insert an analysis_history record, then commit. The browser displays the result and the history page can retrieve it later.

Horizon and output shape

Weekly maps to 13 days; month1, month2 and month3 map to 30, 60 and 90 days. The legacy monthly alias maps to 30 days. Unknown horizon values currently fall back to weekly. The market value is recorded and included in the prompt; it does not select an independent Thai-market data adapter. SET support therefore remains provider-dependent and incomplete.

The active analysis endpoint returns a mixed JSON contract: AI fields use currentPrice, priceTarget, summaryTH, optionStrategy, callExplanation, putExplanation, sentimentScores, factors, keyNews and priceScenarios; backend enrichment uses option_levels, days_left, stock_data and model_used. The legacy summaryTH field now requests an English summary. The frontend consumes these active names rather than the unused snake_case AnalysisResponse schema.

Failure behavior and improvements

Missing credentials or provider failures produce HTTP errors; AI JSON is extracted from the first opening to the last closing brace and parsed. Optional Finnhub sentiment/earnings failures degrade to missing context. There is no schema-enforced AI response, retry policy or queue. Broad exception handling can turn lower-level errors into HTTP 500, so stricter validation and safer error mapping are recommended.

DESIGN DOCUMENT / 05

Database model and relationships



Figure 2. SQLAlchemy entities. subscriptions, analysis_history and daily_usage each reference users.id.

Table	Fields and purpose
users	id PK; email unique/indexed/non-null; password_hash; name; role enum free/pro; created_at. Identity and current access tier.
subscriptions	id PK; user_id FK/indexed; status pending/approved; plan; amount; slip_note; approved_at; expires_at; created_at. Manual membership request and expiry record.
analysis_history	id PK; user_id FK/indexed; ticker; market; direction; confidence; result JSON; created_at. Full result snapshots for the latest-history view.
daily_usage	id PK; user_id FK/indexed; date string YYYY-MM-DD; count. Counts successful analyses used by the Free quota.
flash_sales	id PK; original_price; sale_price; discount_pct; label; started_at; ends_at; is_active. Time-bounded membership price overrides.

Integrity and lifecycle

Tables are created with Base.metadata.create_all at backend import/startup. A database session is opened per request and closed in a finally block. Analysis history and usage changes are committed together. Approval marks the subscription approved and assigns 30 days of Pro; authenticated user resolution checks that an approved, unexpired record exists.

Important implementation gap: daily_usage expresses an intended user/date uniqueness rule in an ad-hoc __table_args__ dictionary, not a SQLAlchemy UniqueConstraint. This document does not claim the database enforces that rule. A proper constraint plus an atomic upsert is required to avoid duplicate counts or quota races under concurrent requests. Foreign keys exist, but cascade-deletion and data-retention policies are not specified in the models.

DESIGN DOCUMENT / 06

API interface design

Access legend: Public = no user token; User = bearer token; Pro = user token plus active Pro for gated categories; Admin = configured X-Admin-Password header. One admin list route also requires a user token.

Method / endpoint	Access	Purpose
GET /health	Public	Service response + credential-configured flags; not a provider-call test.
POST /api/auth/register	Public	Validate email/password and create a Free user; return token/user.
POST /api/auth/login	Public	Verify password and issue token.
GET /api/auth/me	User	Return profile, role and usage_today.
POST /api/analyze	User	Fetch context, request AI analysis and save a result.
GET /api/history	User	Latest 20 analyses belonging to the caller.
GET /api/scanner/categories	User	Return watchlists and locked flags.
POST /api/scanner/scan	User / Pro	Scan category; reject gated categories for Free users.
GET /api/glossary	Public	Grouped investment terms and examples.
POST /api/subscribe/request	User	Create pending request at current membership price.
GET /api/subscribe/status	User	Latest subscription, price, payment details and role.
GET /api/admin/subscriptions	User + Admin	List pending requests with requester details.
POST /api/admin/approve/{sub_id}	Admin	Approve request and set Pro expiry.
GET /api/flashsale	Public	Active sale or null.
POST /api/admin/flashsale	Admin	Deactivate prior sales and create a timed sale.
DELETE /api/admin/flashsale	Admin	Deactivate flash sales.

Error and authorization contract

Missing or invalid user authentication returns 401. Incorrect admin credentials or a locked Pro scanner category return 403. Missing categories and subscription records can return 404; malformed business input commonly returns 400; exhausted Free analysis quota returns 429 with X-Upgrade=true. Pydantic validates typed request bodies, while scanner input is currently an untyped dictionary. Client pages surface API detail messages to users.

DESIGN DOCUMENT / 07

Screen design and user journeys

Screen	User interaction / information design
Analysis /	Ticker input, US/SET selection and four horizon choices. Result cards show direction/confidence, current and target prices, CALL/PUT explanations, summary and latest news. Detailed sections add strategy, option levels, calculator, scenarios, factors and key news.
Scanner /scanner	Load categories, show locked Pro categories, select a watchlist, then inspect ranked results and their heuristic scores/reasons. Failed provider lookups can reduce result count.
History /history	Display the latest saved analyses for the authenticated user. Open a stored result to inspect the same detailed components without repeating an AI request.
Glossary /glossary	Search terms, definitions and examples across grouped entries. The current backend contains 16 terms, not the larger count claimed by older documentation.
Pro membership /subscribe	Show current role, latest request, configured payment instructions and price. Submit a note/request for manual review. This screen does not verify or process a bank transfer.
Admin /admin	Enter the admin credential, review pending requests, approve membership, and create/cancel timed promotions. Admin credential is held in page state.

Common navigation and state

The shared header provides sign-in/sign-out controls and six navigation links. The navigation bar refreshes the signed-in profile on route changes and shows role plus daily count. Axios adds the token from browser localStorage to API calls. Sign-out removes the local token; the existing JWT is not centrally revoked.

Detailed calculation controls

Strike, days remaining and IV (%) are editable in the AnalysisDetails component. The browser recomputes finish-above / finish-below strike estimates locally. Thai backend level labels are mapped to Conservative, Moderate and Aggressive; recommendation labels are mapped into English. Price scenarios use inline SVG rather than a mandatory live chart feed.

Usability and accessibility considerations

The interface uses a dark theme, grouped cards, loading text and disabled analysis submission while busy. The calculator includes numeric input labels and the scenario graphic includes an accessible description. Further checks should cover keyboard navigation, error focus, narrow-screen metrics and contrast. The analysis metrics and CALL/PUT grids currently have fixed column counts, so full mobile accessibility is not asserted.

DESIGN DOCUMENT / 08

Probability and scanner design

Terminal-price probability model

The implemented approximation uses a lognormal terminal-price assumption with constant volatility and risk-free rate. With current price S, strike K, time T = days/365, IV sigma and r = 0.05, the code computes:

Expression	Meaning
d2 = [ln(S/K) + (r - sigma^2/2)T] / [sigma sqrt(T)]	Standardized log-price term.
P(finish above K) = N(d2)	Normal CDF approximation implemented through erf.
P(finish below K) = N(-d2)	Complement of the finish-above estimate.
Target = S + multiplier x S x sigma x sqrt(T)	Option-level target using multiplier 0.4, 1.0 or 1.6, with sign selected from direction.

These are terminal-price estimates, not probabilities of touching a strike during the interval and not probabilities of profit after premium/fees. IV comes from AI output with a 0.5 fallback, rather than a retrieved option chain. The illustrative payouts of 50%, 100% and 200% are assumptions, and EV = p x payout - (1-p) x 100 is not a quoted or backtested return.

Direction and level behavior

The backend generates upward levels for BULLISH and downward levels for other direction values, including NEUTRAL. The browser presents both finish-above and finish-below figures once it has valid numeric inputs. Invalid/non-positive backend model inputs return a zero-probability N/A result; the frontend asks for valid values instead of showing a number.

Scanner heuristics

Nine predefined watchlists are implemented: three Free and six Pro. Scores combine absolute daily movement, recent news count, analyst bias, proximity to earnings and available sentiment. An option-cost tier uses a rough premium = 5% of stock price and a 100-share contract multiplier. It does not request live premiums, strikes, expiries or Greeks. Results rank observable catalysts, not validated future returns.

Local numerical checks

The submission builder checks complementary above/below probabilities, invalid-input behavior, three generated levels and monotonic above-strike probabilities as K rises. These are offline unit-level checks of the existing formula, not evidence that the model predicts market outcomes.

DESIGN DOCUMENT / 09

Security, access and deployment

Concern	Current design and known boundary
Password handling	bcrypt password hashes are stored rather than plaintext account passwords. Registration currently accepts passwords of six or more characters.
Session credentials	JWTs include user ID and expiry; the configured default expiry is 720 hours. Browser localStorage is convenient but exposes tokens to injected scripts. There is no refresh-token rotation or server-side logout revocation.
Authorization	History is filtered by caller ID; Free quota and Pro scanner gates are enforced by backend logic. Administrative mutations use a shared header credential, not an admin role or a full RBAC system.
Secret configuration	Provider keys and database/JWT credentials are supplied through environment variables. The submission bundle excludes real .env files, database contents and local logs. Example values are configuration placeholders.
Network exposure	Local Compose binds host services to loopback. Server Compose exposes the frontend only. Existing CORS allows all origins and credentials; production should use an explicit origin allowlist and HTTPS.
Provider and AI trust	Provider context can be incomplete and model JSON can be malformed. News text is interpolated into the prompt. Structured-output validation, prompt-input isolation and bounded error responses are improvement items.

Local reproducibility

Install Docker Desktop with Linux containers. Copy .env.example to .env and set fresh POSTGRES_PASSWORD, JWT_SECRET, ADMIN_PASSWORD and the two provider keys. Run docker compose up -d --build. Open http://localhost:3000; inspect http://localhost:8000/docs and /health. Database tables are created at startup. DBeaver can connect to localhost:5432, database stock_analyzer, user postgres, using the configured local database password.

University-server readiness

A remote Ubuntu machine was provided for the group, but remote deployment is not verified in this submission run. Use the server Compose file only after configuring secrets, network access, backups and HTTPS. DBeaver is a database administration client, not the deployment host. The optional form field for a RAI website should stay blank until an accessible deployment has been tested.

DESIGN DOCUMENT / 10

Verification and acceptance plan

Verification date: 24 October 2026. Source review is current; live runtime checks below reflect this run only. Earlier project notes are historical evidence and are not re-labelled as newly passed tests.

Check	Result / evidence
Python source syntax	AST parsing of all backend .py modules is performed by the submission builder. No provider calls or database writes are required.
Probability behavior	Offline checks exercise complement, monotonicity, invalid-input handling and level count. Recorded in verification.json.
Docker service status	UNAVAILABLE in this run: Docker Desktop Linux engine pipe was not available.
Backend /health	UNAVAILABLE in this run: localhost:8000 refused the connection. No live provider validation is claimed.
Frontend route /	UNAVAILABLE in this run: localhost:3000 refused the connection. No fresh browser demo was recorded.
Integration coverage	tools/verify_features.py exists for Free/Pro gating, missing optional sentiment, subscriptions, admin authorization, flash sales, glossary and history. It was inspected, not executed in this run.
Submission contents	A clean active-source bundle, environment example, Docker configuration, design document and manifest are generated. Real secret values are checked locally without printing them.

Acceptance checks before recording the demonstration

ID	Reviewer action and expected outcome
AT-01	Register/sign in; verify wrong-password and missing-token errors, then sign out.
AT-02	Analyze a valid US ticker; check price fields, explanations, news and detailed panels; reopen the stored result in History.
AT-03	Use three Free analyses; the next successful attempt is blocked with 429. Confirm active Pro can continue.
AT-04	Change calculator strike/days/IV and verify both probabilities update; show estimates and assumptions, not guaranteed returns.
AT-05	Scan a Free category; confirm a Pro category is blocked for Free and available after approved Pro membership.
AT-06	Submit a membership note, approve it with admin credentials, then inspect expiry and role. Use test/demo payment data only.
AT-07	Search Glossary, restart containers and verify user/history persistence. Test core flows at desktop and narrow viewport sizes.

DESIGN DOCUMENT / 11

Handover, limitations and next steps

Repository and submission contents

The clean source bundle contains the active backend/, frontend/, Docker Compose files, configuration example, Windows launcher, existing integration-check script and documentation. Legacy Express/vanilla-JS server/ and public/ folders are excluded because the Docker application uses FastAPI and Next.js; db/data.json and all environment secrets are also excluded. The manifest records relative paths, sizes and SHA-256 hashes for traceability.

Verified source repository: https://github.com/Panupon777/stock-option-analyzer. Visibility is Public so the assessor can review and clone it without an invitation. The source release omits real environment files and user databases, removes insecure credential fallbacks, and requires the reviewer to supply their own provider keys and secrets. Published history was rebuilt from the sanitized source. University-server deployment remains unverified.

Known implementation limitations

The application depends on third-party credentials, account entitlements and latency. Scanner requests can take minutes and skip unavailable symbols. The frontend proxy permits long requests but does not provide a background-job queue. Thai-market selection is not backed by a separate data adapter. AI-derived confidence, sentiment and IV are not calibrated estimates, and model identifiers may need updating if provider availability changes.

The release requires database, JWT and admin credentials through environment configuration; insecure credential fallbacks are removed. Production hardening remains necessary: effective user/date uniqueness, atomic quotas, schema-validated AI output, origin restrictions, stronger admin authorization, safe errors, rate limiting, dependency maintenance and migrations. PostgreSQL persistence does not itself provide a backup or disaster-recovery policy. Payment is manual and no broker orders are placed.

Recommended development priorities

Priority	Next step
1. Reliable demo	Start Docker, complete AT-01 through AT-07, capture evidence and configure a repository the assessor can access.
2. Data correctness	Validate AI JSON, distinguish provider facts from generated estimates, and handle unsupported symbols/markets explicitly.
3. Integrity and security	Add migrations and true constraints; enforce atomic quotas; replace shared admin credentials with roles; add origin restrictions and HTTPS.
4. Evaluation	Add deterministic provider mocks and CI, then evaluate model outputs against a defined historical dataset without claiming profit guarantees.

Implementation references

Primary evidence: backend/main.py, auth.py, database.py, schemas.py, config.py, finnhub_client.py, probability.py, scanner.py and glossary.py; frontend/app/, frontend/components/ and frontend/lib/api.ts; docker-compose.yml, docker-compose.server.yml and tools/verify_features.py. Older README migration claims were reconciled against these files rather than copied as test results.